

MiniEnglish_To_Nepali

March 24, 2026

Task: English $\rightarrow$ Nepali Translation using a Transformer.

Input: English sentence (sequence of words)

Output: Corresponding Nepali sentence

Approach: Sequence-to-sequence Transformer (encoder-decoder)

Dataset: 150 English-Nepali sentence pairs (toy dataset)

Goal: Learn the mapping from English words to Nepali words using attention mechanisms.

```
[2]: # New sentences to append
english_sentences = [
    "I am happy", "You are welcome", "He is my brother", "She is my sister", "\u2192",
    "\u2192We are learning",
    "They are studying", "It is cold today", "I like chocolate", "Do you like", "\u2192",
    "\u2192coffee", "Where is the library",
    "I am going to school", "Please open the book", "Close the door", "I have a", "\u2192",
    "\u2192cat", "He is reading",
    "She is writing", "We are playing", "They are singing", "It is very hot", "\u2192",
    "\u2192I need a pen",
    "I am feeling tired", "You are very kind", "He is strong", "She is smart", "\u2192",
    "\u2192We are ready",
    "They are running", "It is raining heavily", "I love music", "Do you know", "\u2192",
    "\u2192him?", "Where are you going?",
    "I am waiting", "Please help me", "Open the window", "Close the book", "I", "\u2192",
    "\u2192like tea",
    "He is cooking", "She is cleaning", "We are watching TV", "They are", "\u2192",
    "\u2192dancing", "It is sunny",
    "I am learning Python", "You are amazing", "He is busy", "She is talking", "\u2192",
    "\u2192We are eating",
    "They are laughing", "It is windy", "I want coffee", "Do you want water?", "\u2192",
    "\u2192Where is the shop?",
    "I am tired today", "Please sit here", "He is sleeping now", "She is", "\u2192",
    "\u2192working", "We are going out",
    "They are listening", "It is snowing", "I like reading", "Do you like", "\u2192",
    "\u2192movies?", "Where do you live?",
    "I am going home", "Please come here", "Open the bag", "Close the window", "\u2192",
    "\u2192I love my family",
```


Total English sentences: 100
 Total Nepali sentences: 100
 Do you speak English? --> ?
 Where is the market? --> ?
 I am happy now -->
 Please help me now -->
 He is running fast -->

Problem

- Transformers cannot directly work with text, so they require numbers as input.
- How to ensure all sentence batch have same length.
- How to make start and end of sentence in decoder.

Solution

- Convert each word to ID(tokenization \$ vocabulary)
- Use Padding to ensure same length
- Add special tokens :
 - <pad> → 0 (for padding)
 - <sos> → 1 (start of sentence)
 - <eos> → 2 (end of sentence)

```
[3]: # 2(a) Vocabulary Creation

# Special tokens
PAD_TOKEN = "<pad>"
SOS_TOKEN = "<sos>"
EOS_TOKEN = "<eos>"

# Function to create vocabulary from a list of sentences
def build_vocab(sentences):
    vocab = {PAD_TOKEN: 0, SOS_TOKEN: 1, EOS_TOKEN: 2} # start with special
    ↪tokens
    idx = 3 # start indexing words from 3
    for sentence in sentences:
        for word in sentence.split(): # simple split by space
            if word not in vocab:
                vocab[word] = idx
                idx += 1
    return vocab

# Create vocabularies
src_vocab = build_vocab(english_sentences) # English
tgt_vocab = build_vocab(nepali_sentences) # Nepali

# Inverse vocab for decoding later (ID -> word)
inv_src_vocab = {i: w for w, i in src_vocab.items()}
```

```

inv_tgt_vocab = {i: w for w, i in tgt_vocab.items()}

# Print some info
print("English vocab size:", len(src_vocab))
print("Nepali vocab size:", len(tgt_vocab))
print("\nSample English vocab:", list(src_vocab.items())[:10])
print("\nSample Nepali vocab:", list(tgt_vocab.items())[:10])

```

English vocab size: 119

Nepali vocab size: 135

Sample English vocab: [('<pad>', 0), ('<sos>', 1), ('<eos>', 2), ('I', 3), ('am', 4), ('happy', 5), ('You', 6), ('are', 7), ('welcome', 8), ('He', 9)]

Sample Nepali vocab: [('<pad>', 0), ('<sos>', 1), ('<eos>', 2), (' ', 3), (' ', 4), (' ', 5), (' ', 6), (' ', 7), (' ', 8), (' ', 9)]

```

[4]: # 2b. Tokenize and Pad sentence
import torch
from torch.nn.utils.rnn import pad_sequence

# Function to encode a sentence into token IDs
def encode_sentence(sentence, vocab, add_sos_eos=False):
    tokens = [vocab[word] for word in sentence.split()]
    if add_sos_eos:
        tokens = [vocab[SOS_TOKEN]] + tokens + [vocab[EOS_TOKEN]]
    return torch.tensor(tokens, dtype=torch.long)

# Encode all English sentences (source)
src_encoded = [encode_sentence(s, src_vocab) for s in english_sentences]

# Encode all Nepali sentences (target) with <sos> and <eos>
tgt_encoded = [encode_sentence(s, tgt_vocab, add_sos_eos=True) for s in ↵
    ↵nepali_sentences]

# Pad sequences so all sentences have same length
src_padded = pad_sequence(src_encoded, batch_first=True, ↵
    ↵padding_value=src_vocab[PAD_TOKEN])
tgt_padded = pad_sequence(tgt_encoded, batch_first=True, ↵
    ↵padding_value=tgt_vocab[PAD_TOKEN])

# Print shapes and some examples
print("Source padded shape:", src_padded.shape)
print("Target padded shape:", tgt_padded.shape)

print("\nFirst English sentence IDs:", src_padded[0])
print("First Nepali sentence IDs:", tgt_padded[0])

```

Source padded shape: torch.Size([100, 5])

Target padded shape: torch.Size([100, 7])

First English sentence IDs: tensor([3, 4, 5, 0, 0])

First Nepali sentence IDs: tensor([1, 3, 4, 5, 2, 0, 0])

Problem

- Transformers cannot directly pass token ID so we need vectors that represent word meaning. Thus
- Convert token IDs to Embedding vectors

- Transformers also need to know word order, because attention alone has no sense of sequence.
- Add positional encoding.

Solution

- Use PyTorch nn.Embedding for source (English) sentences.
- Compute positional encodings using sinusoidal formulas.
- Add positional encodings to embeddings.

Output tensor → [batch_size, seq_len, emb_dim], ready for encoder layers.

```
[5]: # Step 3a: Create Embeddings

import math
import torch.nn as nn

# Parameters
EMB_SIZE = 32 # embedding dimension
BATCH_SIZE = src_padded.shape[0]
SEQ_LEN = src_padded.shape[1]

# Source embedding
src_emb = nn.Embedding(len(src_vocab), EMB_SIZE)

# Get embedded input
embedded_input = src_emb(src_padded) # shape: [batch_size, seq_len, emb_dim]

print("Embedded input shape:", embedded_input.shape)
print("First word embedding of first sentence:\n", embedded_input[0,0])
```

Embedded input shape: torch.Size([100, 5, 32])

First word embedding of first sentence:

```
tensor([-9.1639e-01, -2.4395e-01,  1.6032e-01,  2.1606e-01, -1.4055e-03,
        7.3143e-01, -1.3946e+00, -1.3481e+00,  1.0791e+00,  3.3933e-01,
        1.4593e+00,  1.5259e-01,  1.9597e+00, -1.4282e+00,  3.7937e-01,
        2.9225e-01, -9.2645e-01,  5.2803e-01, -6.0637e-01, -4.5237e-02,
       -2.3975e+00, -2.5463e-01,  9.5361e-01, -4.5007e-01, -1.6177e+00,
       -1.9872e+00,  2.8357e-01,  5.5845e-01, -3.7146e-01,  2.6336e-01,
        1.5130e+00,  1.6269e+00], grad_fn=<SelectBackward0>)
```

```
[6]: # Positional encoding

# Create positional encoding matrix
pos_encoding = torch.zeros(SEQ_LEN, EMB_SIZE)
for pos in range(SEQ_LEN):
    for i in range(0, EMB_SIZE, 2): # even dimensions
        pos_encoding[pos, i] = math.sin(pos / (10000 ** (i / EMB_SIZE)))
        if i+1 < EMB_SIZE:
            pos_encoding[pos, i+1] = math.cos(pos / (10000 ** (i / EMB_SIZE)))

# Expand to batch size
pos_encoding = pos_encoding.unsqueeze(0).repeat(BATCH_SIZE, 1, 1)

# Add positional encoding to embeddings
embedded_with_pos = embedded_input + pos_encoding

print("Embedded input with positional encoding shape:", embedded_with_pos.shape)
print("First word embedding with positional encoding for first sentence:\n",
      ↪embedded_with_pos[0,0])
```

```
Embedded input with positional encoding shape: torch.Size([100, 5, 32])
First word embedding with positional encoding for first sentence:
tensor([-9.1639e-01,  7.5605e-01,  1.6032e-01,  1.2161e+00, -1.4055e-03,
        1.7314e+00, -1.3946e+00, -3.4811e-01,  1.0791e+00,  1.3393e+00,
        1.4593e+00,  1.1526e+00,  1.9597e+00, -4.2818e-01,  3.7937e-01,
        1.2923e+00, -9.2645e-01,  1.5280e+00, -6.0637e-01,  9.5476e-01,
        -2.3975e+00,  7.4537e-01,  9.5361e-01,  5.4993e-01, -1.6177e+00,
        -9.8721e-01,  2.8357e-01,  1.5585e+00, -3.7146e-01,  1.2634e+00,
        1.5130e+00,  2.6269e+00], grad_fn=<SelectBackward0>)
```

4. Multi-Head Self-Attention in the Encoder

Problem

- Words in a sentence are not independent.
- Each word should attend to all other words to understand context
- So, we need
 - i. A mechanism to let each word see other words in the sentence → Self-Attention.
 - ii. Multiple "attention heads" to capture different types of relationships → Multi-Head Attention

Solution

- Apply nn.MultiheadAttention.
- With different Query, Key and Values

```
[7]: #4a Multihead Attention
# Parameters
NHEAD = 4 # number of attention heads
```

```

# PyTorch MultiheadAttention
self_attn = nn.MultiheadAttention(embed_dim=EMB_SIZE, num_heads=NHEAD,
    ↪batch_first=True)

# embedded_with_pos shape: [batch_size, seq_len, emb_dim]
# For self-attention, query=key=value=embedded input
attn_output, attn_weights = self_attn(embedded_with_pos, embedded_with_pos,
    ↪embedded_with_pos)

print("Attention output shape:", attn_output.shape) # [batch_size, seq_len,
    ↪emb_dim]
print("Attention weights shape:", attn_weights.shape) # [batch_size, seq_len,
    ↪seq_len]
print("\nAttention weights for first word in first sentence:\n",
    ↪attn_weights[0,0])

```

Attention output shape: torch.Size([100, 5, 32])

Attention weights shape: torch.Size([100, 5, 5])

Attention weights for first word in first sentence:

tensor([0.1437, 0.2358, 0.1983, 0.1991, 0.2231], grad_fn=<SelectBackward0>)

```

[8]: # Encoder layer
# Parameters
FFN_HID_DIM = 64 # hidden dimension in feed-forward
NUM_LAYERS = 2 # number of encoder layers

# Define one encoder layer
class EncoderLayer(nn.Module):
    def __init__(self, emb_size, nhead, ffn_hid_dim, dropout=0.1):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(embed_dim=emb_size,
    ↪num_heads=nhead, batch_first=True)
        self.norm1 = nn.LayerNorm(emb_size)
        self.norm2 = nn.LayerNorm(emb_size)
        self.ffn = nn.Sequential(
            nn.Linear(emb_size, ffn_hid_dim),
            nn.ReLU(),
            nn.Linear(ffn_hid_dim, emb_size)
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # Multi-head self-attention
        attn_output, attn_weights = self.self_attn(x, x, x)
        x = self.norm1(x + self.dropout(attn_output)) # Residual + LayerNorm

```

```

        # Feed-Forward Network
        ffn_output = self.ffn(x)
        x = self.norm2(x + self.dropout(ffn_output)) # Residual + LayerNorm
        return x

# Stack multiple encoder layers
class Encoder(nn.Module):
    def __init__(self, num_layers, emb_size, nhead, ffn_hid_dim):
        super().__init__()
        self.layers = nn.ModuleList([
            EncoderLayer(emb_size, nhead, ffn_hid_dim) for _ in
↪range(num_layers)
        ])

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

# Instantiate encoder
encoder = Encoder(NUM_LAYERS, EMB_SIZE, NHEAD, FFN_HID_DIM)

# Forward pass through encoder
encoder_output = encoder(embedded_with_pos)

print("Encoder output shape:", encoder_output.shape) # [batch_size, seq_len,
↪emb_dim]
print("First word representation of first sentence:\n", encoder_output[0,0])

```

Encoder output shape: torch.Size([100, 5, 32])

First word representation of first sentence:

```

tensor([-1.8133,  0.5360, -0.0217,  0.8793, -0.1459,  1.0457, -1.8165, -1.0899,
         0.1067,  1.1995,  1.0067,  0.4866,  0.4548, -0.4951,  0.2407,  0.6271,
        -1.0589,  1.3515, -0.7120, -0.1305, -1.8448,  0.1497,  0.5045,  0.2680,
        -1.7167, -1.2675, -0.4971,  1.2769,  0.2031, -0.0289,  0.1269,  2.1753],
        grad_fn=<SelectBackward0>)

```

[9]: # decoder

```

class DecoderLayer(nn.Module):
    def __init__(self, emb_size, nhead, ffn_hid_dim, dropout=0.1):
        super().__init__()
        # Masked self-attention for target sequence
        self.self_attn = nn.MultiheadAttention(embed_dim=emb_size,
↪num_heads=nhead, batch_first=True)
        # Attention over encoder output

```

```

        self.enc_attn = nn.MultiheadAttention(embed_dim=emb_size,
        ↪ num_heads=nhead, batch_first=True)
        # Feed-forward network
        self.ffn = nn.Sequential(
            nn.Linear(emb_size, ffn_hid_dim),
            nn.ReLU(),
            nn.Linear(ffn_hid_dim, emb_size)
        )
        # Layer norms
        self.norm1 = nn.LayerNorm(emb_size)
        self.norm2 = nn.LayerNorm(emb_size)
        self.norm3 = nn.LayerNorm(emb_size)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, enc_out, tgt_mask=None):
        # Masked self-attention
        attn_output, _ = self.self_attn(x, x, x, attn_mask=tgt_mask)
        x = self.norm1(x + self.dropout(attn_output))

        # Encoder-decoder attention
        attn_output, _ = self.enc_attn(x, enc_out, enc_out)
        x = self.norm2(x + self.dropout(attn_output))

        # Feed-forward
        ffn_output = self.ffn(x)
        x = self.norm3(x + self.dropout(ffn_output))

        return x

# Decoder stack
class Decoder(nn.Module):
    def __init__(self, num_layers, emb_size, nhead, ffn_hid_dim):
        super().__init__()
        self.layers = nn.ModuleList([DecoderLayer(emb_size, nhead, ffn_hid_dim)
        ↪ for _ in range(num_layers)])

    def forward(self, x, enc_out, tgt_mask=None):
        for layer in self.layers:
            x = layer(x, enc_out, tgt_mask)
        return x

```

```

[10]: # Look_Ahead Mask(Decoder)
# Function to create look-ahead mask
def create_look_ahead_mask(size):
    mask = torch.triu(torch.ones(size, size), diagonal=1).bool() # upper
    ↪ triangular
    return mask

```

```

[11]: class TransformerModel(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size, emb_size, nhead,
        ↪ffn_hid_dim, num_layers):
        super().__init__()
        # Embeddings
        self.src_emb = nn.Embedding(src_vocab_size, emb_size)
        self.tgt_emb = nn.Embedding(tgt_vocab_size, emb_size)

        # Positional encoding
        self.pos_encoding = self.create_positional_encoding(100, emb_size) # ↪
        ↪max seq len 100

        # Encoder and decoder
        self.encoder = Encoder(num_layers, emb_size, nhead, ffn_hid_dim)
        self.decoder = Decoder(num_layers, emb_size, nhead, ffn_hid_dim)

        # Output linear layer
        self.fc_out = nn.Linear(emb_size, tgt_vocab_size)

    def create_positional_encoding(self, max_len, emb_size):
        pos_encoding = torch.zeros(max_len, emb_size)
        for pos in range(max_len):
            for i in range(0, emb_size, 2):
                pos_encoding[pos, i] = math.sin(pos / (10000 ** (i / emb_size)))
                if i+1 < emb_size:
                    pos_encoding[pos, i+1] = math.cos(pos / (10000 ** (i / ↪
        ↪emb_size)))
        return pos_encoding.unsqueeze(0) # [1, max_len, emb_size]

    def forward(self, src, tgt):
        batch_size, src_len = src.shape
        batch_size, tgt_len = tgt.shape

        # Encoder
        src_emb = self.src_emb(src) + self.pos_encoding[:, :src_len, :]
        enc_out = self.encoder(src_emb)

        # Decoder
        tgt_emb = self.tgt_emb(tgt) + self.pos_encoding[:, :tgt_len, :]
        look_ahead_mask = create_look_ahead_mask(tgt_len).to(tgt.device)
        dec_out = self.decoder(tgt_emb, enc_out, tgt_mask=look_ahead_mask)

        # Project to vocabulary
        out = self.fc_out(dec_out) # [batch, tgt_len, tgt_vocab_size]
        return out

```

```
[12]: SRC_VOCAB_SIZE = len(src_vocab)
      TGT_VOCAB_SIZE = len(tgt_vocab)

      model = TransformerModel(SRC_VOCAB_SIZE, TGT_VOCAB_SIZE, EMB_SIZE, NHEAD,
                               ↪FFN_HID_DIM, NUM_LAYERS)

      # Test forward pass
      sample_src = src_padded[:2] # first 2 sentences
      sample_tgt = tgt_padded[:2, :-1] # input to decoder (without last token)

      output = model(sample_src, sample_tgt)
      print("Transformer output shape:", output.shape) # [batch, tgt_len,
                                                         ↪tgt_vocab_size]
```

Transformer output shape: torch.Size([2, 6, 135])

```
[13]: # Preparing for training
      import torch.nn as nn
      import torch.optim as optim

      # Target input and expected output
      tgt_input = tgt_padded[:, :-1] # all tokens except last (<eos>)
      tgt_output = tgt_padded[:, 1:] # all tokens except first (<sos>)

      # Define loss and optimizer
      criterion = nn.CrossEntropyLoss(ignore_index=tgt_vocab[PAD_TOKEN])
      optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
[14]: EPOCHS = 500

      for epoch in range(EPOCHS):
          model.train()
          optimizer.zero_grad()

          # Forward pass
          output = model(src_padded, tgt_input) # [batch, tgt_len, vocab_size]

          # Reshape for loss: [batch * tgt_len, vocab_size] vs [batch * tgt_len]
          loss = criterion(output.reshape(-1, TGT_VOCAB_SIZE), tgt_output.reshape(-1))

          # Backpropagation
          loss.backward()
          optimizer.step()

          if (epoch+1) % 10 == 0:
              print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

```
print("Training finished!")
```

```
Epoch 10, Loss: 4.1957
Epoch 20, Loss: 3.8920
Epoch 30, Loss: 3.5288
Epoch 40, Loss: 3.1774
Epoch 50, Loss: 2.8118
Epoch 60, Loss: 2.4652
Epoch 70, Loss: 2.1414
Epoch 80, Loss: 1.8627
Epoch 90, Loss: 1.6031
Epoch 100, Loss: 1.3980
Epoch 110, Loss: 1.1976
Epoch 120, Loss: 1.0347
Epoch 130, Loss: 0.8872
Epoch 140, Loss: 0.7665
Epoch 150, Loss: 0.6527
Epoch 160, Loss: 0.5658
Epoch 170, Loss: 0.4873
Epoch 180, Loss: 0.4213
Epoch 190, Loss: 0.3626
Epoch 200, Loss: 0.3185
Epoch 210, Loss: 0.2740
Epoch 220, Loss: 0.2429
Epoch 230, Loss: 0.2161
Epoch 240, Loss: 0.1890
Epoch 250, Loss: 0.1682
Epoch 260, Loss: 0.1527
Epoch 270, Loss: 0.1313
Epoch 280, Loss: 0.1208
Epoch 290, Loss: 0.1121
Epoch 300, Loss: 0.0970
Epoch 310, Loss: 0.0899
Epoch 320, Loss: 0.0850
Epoch 330, Loss: 0.0771
Epoch 340, Loss: 0.0722
Epoch 350, Loss: 0.0663
Epoch 360, Loss: 0.0605
Epoch 370, Loss: 0.0594
Epoch 380, Loss: 0.0560
Epoch 390, Loss: 0.0500
Epoch 400, Loss: 0.0490
Epoch 410, Loss: 0.0459
Epoch 420, Loss: 0.0434
Epoch 430, Loss: 0.0403
Epoch 440, Loss: 0.0378
Epoch 450, Loss: 0.0366
Epoch 460, Loss: 0.0369
```

Epoch 470, Loss: 0.0339
Epoch 480, Loss: 0.0316
Epoch 490, Loss: 0.0313
Epoch 500, Loss: 0.0294
Training finished!

```
[18]: def translate(model, sentence, src_vocab, tgt_vocab, inv_tgt_vocab, max_len=20):
    model.eval()

    # Encode source sentence
    src_tokens = torch.tensor([src_vocab[word] for word in sentence.split()]).
    ↪unsqueeze(0) # [1, seq_len]

    # Initialize target sequence with <sos>
    tgt_tokens = torch.tensor([[tgt_vocab[SOS_TOKEN]]], dtype=torch.long)

    output_sentence = []

    for _ in range(max_len):
        # Forward pass
        out = model(src_tokens, tgt_tokens)

        # Get last token probabilities
        next_token = out[0, -1, :].argmax(-1).item()

        # Stop if <eos> is predicted
        if next_token == tgt_vocab[EOS_TOKEN]:
            break

        # Append word to output
        output_sentence.append(inv_tgt_vocab[next_token])

        # Append token to decoder input
        tgt_tokens = torch.cat([tgt_tokens, torch.tensor([[next_token]]),
    ↪dim=1)

    return " ".join(output_sentence)
```

```
[19]: # Example translation
english_test = "I am happy"
nepali_translation = translate(model, english_test, src_vocab, tgt_vocab,
    ↪inv_tgt_vocab)
print("English:", english_test)
print("Predicted Nepali:", nepali_translation)
```

English: I am happy
Predicted Nepali:

```
[23]: test_sentences = [

    "You are welcome", "I love music", "Do you know him?", "Where are you going?
    ↪",
    "I am waiting", "Please help me", "Open the window", "Close the book", "I
    ↪like tea",
    "He is cooking", "She is cleaning", "We are watching TV", "They are
    ↪dancing", "It is sunny",
    "I am learning Python", "You are amazing", "He is busy", "She is talking",
    ↪"We are eating",
    "They are laughing", "It is windy", "I want coffee", "Do you want water?",
    ↪"Where is the shop?",
    "I am tired today", "Please sit here", "He is sleeping now", "She is
    ↪working"

]

for sent in test_sentences:
    translation = translate(model, sent, src_vocab, tgt_vocab, inv_tgt_vocab)
    print(f"English: {sent}")
    print(f"Predicted Nepali: {translation}")
    print("-" * 50)
```

English: You are welcome

Predicted Nepali:

English: I love music

Predicted Nepali:

English: Do you know him?

Predicted Nepali: ?

English: Where are you going?

Predicted Nepali: ?

English: I am waiting

Predicted Nepali:

English: Please help me

Predicted Nepali:

English: Open the window

Predicted Nepali:

English: Close the book

Predicted Nepali:

English: I like tea

Predicted Nepali:

English: He is cooking

Predicted Nepali:

English: She is cleaning

Predicted Nepali:

English: We are watching TV

Predicted Nepali:

English: They are dancing

Predicted Nepali:

English: It is sunny

Predicted Nepali:

English: I am learning Python

Predicted Nepali:

English: You are amazing

Predicted Nepali:

English: He is busy

Predicted Nepali:

English: She is talking

Predicted Nepali:

English: We are eating

Predicted Nepali:

English: They are laughing

Predicted Nepali:

English: It is windy

Predicted Nepali:

English: I want coffee

Predicted Nepali:

English: Do you want water?

Predicted Nepali: ?

English: Where is the shop?

Predicted Nepali: ?

English: I am tired today

Predicted Nepali:

English: Please sit here

Predicted Nepali:

English: He is sleeping now

Predicted Nepali:

English: She is working

Predicted Nepali:

[]:

[]: